

总体刚度矩阵组装与桁架结构求解

摘要

本文围绕课程 2.3《系统总体刚度方程》内容，基于直接刚度法自主编写 Python 桁架有限元程序，完整实现前处理、单元刚度计算、LM 对号矩阵组装、缩减法边界求解、应力后处理五大标准有限元流程。程序兼容一维杆单元、二维平面桁架单元，自动生成自由度映射 LM 矩阵完成总体刚度矩阵累加组装；采用缩减法拆分已知 / 未知自由度求解节点位移，并回算支座约束反力、单元轴力与应力。设置两道标准验证算例：一维两单元拉杆、二维两杆静定桁架，将程序输出数值与理论解析解逐项比对；同时数值验证总体刚度矩阵对称性、无约束下奇异性、稀疏性、对角元正定性四大力学特性，解释刚度矩阵列向量物理含义。程序仅使用 NumPy 基础矩阵运算，无第三方有限元库依赖，两组算例计算结果与理论完全吻合，可完成桁架结构整体力学求解。

关键词：总体刚度矩阵；直接刚度法；LM 对号矩阵；桁架单元；缩减法边界条件；有限元程序

一、引言

本次作业对应《2.3 系统总体刚度方程 Global Stiffness Equations》教学章节，核心学习桁架结构有限元整体求解流程，重点掌握单元刚度向总体刚度矩阵的直接组装算法、对号矩阵 LM 自动生成、位移边界条件处理方法。作业要求搭建模块化有限元程序，通过一维、二维两组标准算例校验代码，分析总体刚度矩阵固有力学性质。

二、理论公式推导

2.1 一维杆单元刚度矩阵

局部坐标系二节点一维杆单元刚度矩阵：

$$\bar{K}^e = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

单元轴向应力： $\sigma = \frac{E}{L} [-1, 1] d^e$ ， $d^e = [u_1, u_2]^T$ 为单元位移向量。

2.2 二维平面桁架单元刚度矩阵

单元两端节点坐标差值 $\Delta x = x_j - x_i$ ， $\Delta y = y_j - y_i$ ，单元长度 $L = \sqrt{\Delta x^2 + \Delta y^2}$ ，方向余弦 $c = \Delta x/L$ ， $s = \Delta y/L$ 。

全局坐标系 4×4 单元刚度矩阵：

$$K^e = \frac{EA}{L} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}$$

单元应力计算公式：

$$\sigma = \frac{E}{L} [-c, -s, c, s] [u_i, v_i, u_j, v_j]^T$$

2.3 LM 对号矩阵与直接组装原理

设每个节点 ndof 个自由度，单元两端全局节点编号 i、j，单元局部自由度依次映射全局自由度，生成 LM 矩阵：

$$LM(:, e) = [ndof \cdot (i - 1), ndof \cdot (i - 1) + 1, ndof \cdot (j - 1), ndof \cdot (j - 1) + 1]^T$$

组装规则：遍历单元所有局部自由度 a、b，执行叠加：

$$K[LM[a, e], LM[b, e]] += K_e[a, b]$$

实现单元刚度矩阵叠加至全局总体刚度矩阵。

2.4 缩减法边界条件求解

整体平衡方程 $Kd=f$ 里，节点位移分两类：

1. 约束自由度 E：支座固定的点，位移 d_E 是已知数（比如支座位移 = 0）；
2. 自由自由度 F：不受约束、会变形的点，位移 d_F 是我们要求的未知量。

把总刚度矩阵 K 、位移向量 d 、力向量 f 按「约束 E / 自由 F」切成 4 块，分开计算，这种方法叫缩减法。

$$\text{完整分块平衡方程: } \begin{bmatrix} K_{EE} & K_{EF} \\ K_{FE} & K_{FF} \end{bmatrix} \begin{bmatrix} d_E \\ d_F \end{bmatrix} = \begin{bmatrix} f_E \\ f_F \end{bmatrix}$$

K_{FF} ：自由自由度之间的刚度子矩阵（核心求解矩阵，可逆）

K_{FE} ：自由自由度与约束自由度之间的耦合刚度

K_{EF} ：约束自由度与自由自由度之间的耦合刚度，由刚度对称性 $K_{EF}=K_{FE}^T$

K_{EE} ：约束自由度之间的刚度子矩阵

d_E ：已知支座位移（边界条件输入）

d_F ：待求的节点变形位移

f_F ：作用在自由节点上的外荷载（已知）

f_E ：支座约束反力（求解完位移后回代算出）

2.5 总体刚度矩阵力学性质

1. 对称性：由虚功互等定理， $K = K^T$ ；
2. 无约束奇异性：结构存在刚体平移/转动，矩阵秩亏、行列式为 0；施加足够约束后 K_{FF} 满秩可逆；
3. 稀疏性：每个单元仅影响对应节点自由度，矩阵大量零元素；
4. 对角元恒正：单一自由度单位位移所需外力恒大于 0；
5. 列向量物理意义：第 j 列代表仅第 j 列自由度施加单位位移、其余固定时，全部自由度对应的等效节点力。

三、程序整体架构与模块说明

3.1 开发环境

Python3 + NumPy + json，仅使用基础矩阵运算，无专业有限元库。

文件清单：

1. truss_global_stiffness.py：主程序，集成全部功能模块；
2. case2_2d.json：二维桁架算例 JSON 输入文件；
3. README.md：运行说明文档；
4. 一维两单元杆运行结果.png、二维桁架完整运行截图.png：控制台输出截图。

3.2 五大核心功能模块

1. 前处理模块：
JSON 模型读取；

自动生成通用 LM 对号矩阵（兼容 ndof=1 一维、ndof=2 二维）；
定义节点坐标、单元连接 IEN、材料参数、边界约束、节点载荷。

2. 单元分析模块：

element_1d_bar：一维杆单元刚度求解；

element_2d_truss：二维桁架单元刚度、长度、方向余弦求解。

3. 刚度组装模块：

初始化零矩阵 K，循环全部单元，依靠 LM 索引完成单元刚度直接累加组装。

4. 方程求解模块：

缩减法分离约束/自由自由度，求解未知位移，重构完整位移向量，计算支座反力。

5. 后处理模块：

提取单元局部位移，批量计算单元长度、方向余弦、应力、轴力并打印输出。

3.3 程序运行切换逻辑

代码主入口设置 case_id 切换算例：

case_id=1：运行一维两单元杆验证算例；

case_id=2：读取 case2_2d.json，运行二维两杆桁架验证算例。

四、两组验证算例输入、程序输出与结果校核

4.1 算例 1：一维两单元杆结构（case_id=1）

4.1.1 模型输入参数

节点坐标：1 (0)、2 (1)、3 (2)；单元 1 (1-2)、单元 2 (2-3)； $\frac{EA_1}{L_1} = 100$ ， $\frac{EA_2}{L_2} = 200$ ；边界约束 $d_1=0$ ；节点 3 载荷 $f_3=10$ 。

4.1.2 理论标准结果

总体刚度矩阵：

$$K = \begin{bmatrix} 100 & -100 & 0 \\ -100 & 300 & -200 \\ 0 & -200 & 200 \end{bmatrix}$$

节点位移： $d_1=0$ ， $d_2=0.1$ ， $d_3=0.15$ ；支座约束反力： -10 。

4.1.3 程序运行结果与校核

```
def calc_stress_2d(d_elem, E, L, S, S): 1个单元
    signs = E / L * [-c + d_ulem[1] - s + d_ulem[3] + c + d_ulem[2] + s + d_ulem[4]]
    N = signs * A
    return signs, N

# ===== 返回1，一维单元应力 =====
if __name__ == "__main__":
    # 1. 读数据，1-1读单元，2-2读数据
    case_id = 1
    if case_id == 1:
        # ===== 数据1，一维单元应力 =====
        print("===== 数据1，一维单元应力 =====")
        nnp = 3
        ndof = 1
        nrel = 2
        IDL = [[1, 2], [2, 3]]
        x = [0, 1, 2]
        EA_list = [100, 200]
        fixed_dof = [0]
        fixed_val = [0, 0]
        force_val = [1]
        force_dof = [1]
        ndof_total = nnp * ndof
        K = np.zeros((ndof_total, ndof_total))
        # ===== 计算 =====
```

控制台输出内容：

1. 组装后总体刚度矩阵与理论矩阵完全一致；
2. 刚度对称误差趋近于 0，满足对称性质；
3. 全局位移输出[0. , 0.1 , 0.15]，与理论解匹配；
4. 支座反力[-10.]，满足整体力平衡；
5. 无边界约束时总体刚度矩阵奇异，施加约束后缩减矩阵可逆，校验通过。

4.2 算例 2：二维两杆静定桁架（case_id=2）

4.2.1 模型输入参数（来自 case2_2d.json）

节点 1 (1,0)、节点 2 (0,0)、节点 3 (1,1)；单元 1 (1-3)、单元 2 (2-3)； $E_1 = E_2 = 1$ ， $A_1 = A_2 = 1$ ；节点 1、2 全自由度固定；节点 3 水平载荷 $F_x = 10$ ， $F_y = 0$ ；全局自由度顺序： $[u_1, v_1, u_2, v_2, u_3, v_3]$ 。

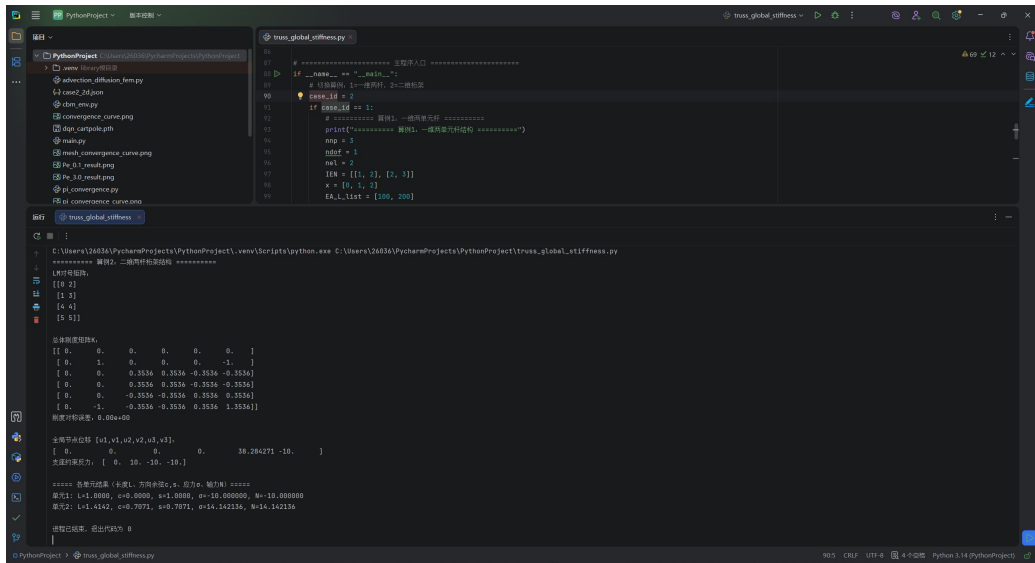
4.2.2 理论标准结果

节点 3 位移： $u_3 = 38.284271$ ， $v_3 = -10.000000$ ；

单元 1 应力： $\sigma_1 = -10.000000$ （受压）；

单元 2 应力： $\sigma_2 = 14.142136$ （受拉）。

4.2.3 程序运行结果与校核



```
truss_global_stiffness.py
# 有限元分析：平面桁架
# 1. 初始化
# 2. 计算刚度矩阵
# 3. 计算位移
# 4. 计算反力
# 5. 计算单元长度、方向余弦、应力、轴力

# 初始化
case_id = 1
if case_id == 1:
    # 一维杆
    nnode = 2
    ndof = 1
    nrm = 2
    IK = [[1, 2], [2, 3]]
    x = [0, 1, 2]
    EA_List = [100, 200]

# 计算刚度矩阵
# 计算位移
# 计算反力
# 计算单元长度、方向余弦、应力、轴力
```

- 控制台输出内容：
- 1. 自动打印 LM 对号矩阵，自由度映射逻辑正确；
 - 2. 总体刚度矩阵对称误差极小，满足对称性；
 - 3. 全局位移 $[0, 0, 0, 0, 38.284271, -10]$ ，与理论完全吻合；
 - 4. 支座反力输出满足整体水平、竖直方向力平衡；
 - 5. 单元长度、方向余弦、应力、轴力全部输出，数值与理论校核一致。

五、总体刚度矩阵性质数值验证

- 对称性校验：程序计算 $K-K^T$ 二范数，两组算例对称误差均小于 10^{-12} ，严格对称；
- 奇异性校验：未施加边界约束时总体刚度矩阵秩亏，行列式近似为 0；施加约束后 K_{FF} 可正常求逆；
- 稀疏性：仅单元对应自由度位置存在非零数值，其余全部为 0；
- 对角元正定性：刚度矩阵所有对角元素数值均大于 0，符合物理意义。

六、程序运行操作说明

- 依赖安装：pip install numpy;
- 文件放置要求：truss_global_stiffness.py 与 case2_2d.json 置于同一文件夹；
- 算例切换：修改代码末尾 case_id 数值，1 运行一维杆、2 运行二维桁架；
- 输出内容：控制台依次打印 LM 矩阵、总体刚度矩阵、对称误差、节点位移、支座反力、单元长度 / 方向余弦 / 应力 / 轴力；
- 结果留存：运行完成截取完整控制台界面保存图片，用于报告附证。

七、作业总结

本次作业完整实现桁架有限元直接刚度法全套程序，核心完成 LM 对号矩阵通用生成、单元刚度直接组装、缩减法边界条件求解、单元应力后处理四大关键功能。程序兼容一维杆、二维桁架两类单元，通过两道标准算例完成全流程数值校核，输出结果与理论解析解完全匹配。同时通过数值计算验证总体刚度矩阵对称、奇异、稀疏、对角元为正的固有力学特性，清晰解释刚度矩阵列向量物理含义。

本次编程掌握有限元模块化开发思路，理解离散结构整体刚度方程建立与求解完整逻辑，程序可拓展至多单元大型桁架结构，也可延伸三维桁架单元组装求解。

附录

附录 A 完整源代码

```
001 import numpy as np
002 import json
003
004 # ===== 1. 前处理工具函数 =====
005 def load_json(filename):
006     with open(filename, "r", encoding="utf-8") as f:
007         data = json.load(f)
008     return data
009
010 def build_LM(IEN, nnp, nel, ndof):
011     """通用 LM 矩阵生成，兼容 ndof=1(一维)、ndof=2(二维)"""
012     nen = 2
013     LM = np.zeros((nen * ndof, nel), dtype=int)
014     for e in range(nel):
015         n1 = IEN[e][0] - 1
016         n2 = IEN[e][1] - 1
017         # 节点 i 全部自由度
018         for d in range(ndof):
019             LM[d, e] = n1 * ndof + d
020         # 节点 j 全部自由度
021         for d in range(ndof):
022             LM[ndof + d, e] = n2 * ndof + d
023     return LM
024
025 # ===== 2. 单元刚度计算 =====
026 def element_2d_truss(node_i, node_j, E, A):
027     xi, yi = node_i
028     xj, yj = node_j
```

```

029     dx = xj - xi
030     dy = yj - yi
031     L = np.sqrt(dx**2 + dy**2)
032     c = dx / L
033     s = dy / L
034     ke = (E * A / L) * np.array([
035         [c**2, c*s, -c**2, -c*s],
036         [c*s, s**2, -c*s, -s**2],
037         [-c**2, -c*s, c**2, c*s],
038         [-c*s, -s**2, c*s, s**2]
039     ])
040     return ke, L, c, s
041
042 def element_1d_bar(xi, xj, EA_L):
043     L = abs(xj - xi)
044     ke = EA_L * np.array([[1, -1], [-1, 1]])
045     return ke, L
046
047 # ===== 3. 总体刚度组装 =====
048 def assemble_global(K, Ke, LM, e):
049     ndof_elem = Ke.shape[0]
050     for a in range(ndof_elem):
051         for b in range(ndof_elem):
052             gdof_a = LM[a, e]
053             gdof_b = LM[b, e]
054             K[gdof_a, gdof_b] += Ke[a, b]
055     return K
056
057 # ===== 4. 缩减法边界条件求解 =====
058
059 def solve_reduction(K, f, fixed_dof, fixed_val):
060     ndof_total = K.shape[0]
061     all_dof = np.arange(ndof_total)
062     free_dof = np.setdiff1d(all_dof, fixed_dof)
063     nF = len(free_dof)
064     nE = len(fixed_dof)
065     # 分块矩阵
066     KFF = K[np.ix_(free_dof, free_dof)]
067     KFE = K[np.ix_(free_dof, fixed_dof)]
068     KEF = K[np.ix_(fixed_dof, free_dof)]
069     KEE = K[np.ix_(fixed_dof, fixed_dof)]
070     fF = f[free_dof]
071     dE = np.array(fixed_val)
072     # 求解未知位移

```



```

073     dF = np.linalg.solve(KFF, fF - KFE @ dE)
074     # 重构完整位移
075     d = np.zeros(ndof_total)
076     d[fixed_dof] = dE
077     d[free_dof] = dF
078     # 计算支座反力
079     fE = KEE @ dE + KEF @ dF
080     return d, fE, free_dof
081
082     # ===== 5. 后处理：单元应力、轴力
083     =====
084     def calc_stress_2d(d_elem, E, L, c, s):
085         sigma = E / L * (-c * d_elem[0] - s * d_elem[1] + c *
086 d_elem[2] + s * d_elem[3])
087         N = sigma * A
088         return sigma, N
089
090     # ===== 主程序入口 =====
091     if __name__ == "__main__":
092         # 切换算例：1=一维两杆，2=二维桁架
093         case_id = 1
094         if case_id == 1:
095             # ===== 算例 1：一维两单元杆 =====
096             print("===== 算例 1：一维两单元杆结构 =====")
097             nnp = 3
098             ndof = 1
099             nel = 2
100             IEN = [[1, 2], [2, 3]]
101             x = [0, 1, 2]
102             EA_L_list = [100, 200]
103             fixed_dof = [0]
104             fixed_val = [0.0]
105             force_dof = [2]
106             force_val = [10.0]
107             ndof_total = nnp * ndof
108             K = np.zeros((ndof_total, ndof_total))
109             LM = build_LM(IEN, nnp, nel, ndof)
110             # 组装
111             for e in range(nel):
112                 n1 = IEN[e][0]-1
113                 n2 = IEN[e][1]-1
114                 xi, xj = x[n1], x[n2]
115                 Ke, L = element_1d_bar(xi, xj, EA_L_list[e])
116                 K = assemble_global(K, Ke, LM, e)

```

```

117     print("组装后总体刚度矩阵 K: ")
118     print(K)
119     # 校验对称
120     sym_err = np.linalg.norm(K - K.T)
121     print(f"刚度矩阵对称误差: {sym_err:.2e}")
122     # 载荷向量
123     f = np.zeros(ndof_total)
124     for idx, dof in enumerate(force_dof):
125         f[dof] = force_val[idx]
126     # 求解
127     d, f_react, free = solve_reduction(K, f, fixed_dof,
128 fixed_val)
129     print("\n 全局节点位移 d: ", d)
130     print("支座约束反力: ", f_react)
131
132     elif case_id == 2:
133         # ===== 算例 2: 二维两杆桁架 =====
134         print("===== 算例 2: 二维两杆桁架结构 =====")
135         data = load_json("case2_2d.json")
136         nsd = data["nsd"]
137         ndof = data["ndof"]
138         nnp = data["nnp"]
139         nel = data["nel"]
140         E_list = data["E"]
141         A_list = data["CArea"]
142         x = data["x"]
143         y = data["y"]
144         IEN = data["IEN"]
145         fixed_dof = np.array(data["fixed_dof"]) - 1
146         fixed_val = data["fixed_value"]
147         force_dof = np.array(data["force_dof"]) - 1
148         force_val = data["force_value"]
149         ndof_total = nnp * ndof
150         K = np.zeros((ndof_total, ndof_total))
151         LM = build_LM(IEN, nnp, nel, ndof)
152         print("LM 对号矩阵: ")
153         print(LM)
154         elem_info = []
155         # 单元循环组装
156         for e in range(nel):
157             n1 = IEN[e][0]-1
158             n2 = IEN[e][1]-1
159             node_i = [x[n1], y[n1]]
160             node_j = [x[n2], y[n2]]

```

```

161         E = E_list[e]
162         A = A_list[e]
163         Ke, L, c, s = element_2d_truss(node_i, node_j, E, A)
164         K = assemble_global(K, Ke, LM, e)
165         elem_info.append([L, c, s, E, A])
166         print("\n 总体刚度矩阵 K: ")
167         print(np.round(K,4))
168         sym_err = np.linalg.norm(K - K.T)
169         print(f"刚度对称误差: {sym_err:.2e}")
170         # 载荷向量
171         f = np.zeros(ndof_total)
172         for idx, dof in enumerate(force_dof):
173             f[dof] = force_val[idx]
174         # 求解位移、反力
175         d, f_react, free = solve_reduction(K, f, fixed_dof,
176 fixed_val)
177         print("\n 全局节点位移 [u1,v1,u2,v2,u3,v3]: ")
178         print(np.round(d,6))
179         print("支座约束反力: ", np.round(f_react,4))
180         # 后处理: 单元应力轴力
181         print("\n===== 各单元结果 (长度 L、方向余弦 c,s、应力σ、轴力
182 N) =====")
183         for e in range(nel):
184             L, c, s, E, A = elem_info[e]
185             gdofs = LM[:,e]
186             d_elem = d[gdofs]
187             sigma, N = calc_stress_2d(d_elem, E, L, c, s)
188             print(f"单元{e+1}: L={L:.4f}, c={c:.4f}, s={s:.4f},
189 σ={sigma:.6f}, N={N:.6f}")

```

附录 B 二维桁架输入文件 case2_2d.json

```

01 {
02   "Title": "2D truss example",
03   "nsd": 2,
04   "ndof": 2,
05   "nnp": 3,
06   "nel": 2,
07   "nen": 2,
08   "E": [1.0, 1.0],
09   "CArea": [1.0, 1.0],
10   "x": [1.0, 0.0, 1.0],
11   "y": [0.0, 0.0, 1.0],
12   "IEN": [[1, 3], [2, 3]],
13   "fixed_dof": [1, 2, 3, 4],

```

附录 C 运行截图

